Let's go back to Catalog Service and see how you can run it as a container. There are different ways to achieve that, but here you will use the out-of-the-box integration of Spring Boot with Cloud Native Buildpacks (https://buildpacks.io), a project initiated by Heroku and Pivotal and now hosted by the CNCF. It provides a high-level abstraction for automatically transforming application source code into container images instead of using a low-level `Dockerfile`.

The Paketo Buildpacks (an implementation of the Cloud Native Buildpacks specification) are fully integrated with the Spring Boot Plugin, both for Gradle and Maven. This means that you can containerize your Spring Boot applications without downloading any additional tools, providing any extra dependencies, or writing a Dockerfile.

Chapter 6 will describe how the Cloud Native Buildpacks project works and how you can configure it to containerize your Spring Boot applications. For now, I'll give you a little preview of its features.

Open a Terminal window, navigate to the root folder of your Catalog Service project (`catalog-service`), and run the `bootBuildImage` Gradle task. That's all you need to do to package your application as a container image, using Cloud Native Buildpacks under the hood.

```
$ ./gradlew bootBuildImage
```

WARNING As I write this, the Paketo project is working on adding support for ARM64 images. You can follow the feature's progress on the Paketo Buildpacks project on GitHub: https://github.com/paketo-buildpacks/stacks/issues/51. Until that's complete, you can still use Buildpacks to build containers and run them via Docker Desktop on Apple Silicon computers, but the build process and the application startup phase will be slower than usual. Until official support is added, you can alternatively use the following command, which points to an experimental version of Paketo Buildpacks with ARM64 support: `./gradlew bootBuildImage --builder ghcr.io/thomasvitale/java -builder-arm64`. Be aware that it's experimental and